

Informe inicial de Análisis Estático Inicial (PMD)

YAZID ALEJANDRO SÁNCHEZ SÁNCHEZ

JOSE ALEJANDRO MARTÍNEZ ARIAS

Introducción

El análisis estático se realizó utilizando la herramienta PMD integrada en Eclipse, siguiendo las reglas estándar de Java definidas. El objetivo es identificar problemas de diseño, posibles errores y violaciones de convenciones de código sin necesidad de ejecutar la aplicación.

Estado Inicial

Al realizar el primer escaneo sobre los paquetes presentation y domain, se obtuvieron los siguientes resultados críticos:

Paquetes afectados: presentation y domain.

Element	# Violations	# Violations/KLOC	/iolations/Method	Project
presentation	59	47.3	0.66	HardestGame-V2
VictoryDialog.java	4	22.9	4.00	HardestGame-V2
ConstructorCallsOverridableMethod	4	22.9	4.00	HardestGame-V2
V2GUI.java	6	42.3	0.12	HardestGame-V2
ConstructorCallsOverridableMethod	6	42.3	0.12	HardestGame-V2
SkinSelectionPanel.java	5	56.8	1.25	HardestGame-V2
ConstructorCallsOverridableMethod	5	56.8	1.25	HardestGame-V2
SettingsPanel.java	6	222.2	3.00	HardestGame-V2
ConstructorCallsOverridableMethod	6	222.2	3.00	HardestGame-V2
PlayPanel.java	6	157.9	3.00	HardestGame-V2
ConstructorCallsOverridableMethod	6	157.9	3.00	HardestGame-V2
MainMenuPanel.java	6	146.3	3.00	HardestGame-V2
ConstructorCallsOverridableMethod	6	146.3	3.00	HardestGame-V2
LevelSelectionPanel.java	4	64.5	2.00	HardestGame-V2
ConstructorCallsOverridableMethod	4	64.5	2.00	HardestGame-V2
GamePanel.java	6	25.2	0.67	HardestGame-V2
ConstructorCallsOverridableMethod	6	25.2	0.67	HardestGame-V2
GameOverDialog.java	4	48.8	4.00	HardestGame-V2
ConstructorCallsOverridableMethod	4	48.8	4.00	HardestGame-V2
CustomButton.java	7	318.2	7.00	HardestGame-V2
ConstructorCallsOverridableMethod	7	318.2	7.00	HardestGame-V2
ColorCustomizationPanel.java	5	45.0	1.25	HardestGame-V2
ConstructorCallsOverridableMethod	5	45.0	1.25	HardestGame-V2
domain	5	4.1	0.02	HardestGame-V2
Player.java	1	7.4	0.03	HardestGame-V2
EmptyMethodInAbstractClassShould	1	7.4	0.03	HardestGame-V2
LevelLoader.java	2	11.6	0.20	HardestGame-V2
ClassWithOnlyPrivateConstructorsSh	1	5.8	0.10	HardestGame-V2
FieldNamingConventions	1	5.8	0.10	HardestGame-V2
DOPOPersistence.java	2	200.0	1.00	HardestGame-V2
AvoidFileStream	2	200.0	1.00	HardestGame-V2

presentation	59
VictoryDialog.java	4
ConstructorCallsOverridableMethod	4
V2GUI.java	6
ConstructorCallsOverridableMethod	6
SkinSelectionPanel.java	5
ConstructorCallsOverridableMethod	5
SettingsPanel.java	6
ConstructorCallsOverridableMethod	6
PlayPanel.java	6
ConstructorCallsOverridableMethod	6
MainMenuPanel.java	6
ConstructorCallsOverridableMethod	6
LevelSelectionPanel.java	4
ConstructorCallsOverridableMethod	4
GamePanel.java	6
ConstructorCallsOverridableMethod	6
GameOverDialog.java	4
ConstructorCallsOverridableMethod	4
CustomButton.java	7
ConstructorCallsOverridableMethod	7
ColorCustomizationPanel.java	5
ConstructorCallsOverridableMethod	5

Regla Violada: ConstructorCallsOverridableMethod

- **Explicación del Error:** Este es un fallo de diseño en la jerarquía de clases de Java. El PMD detectó que en los constructores de casi todos los paneles (ej. PlayPanel, GamePanel, V2GUI) se están llamando métodos públicos. Si una clase hereda de estos paneles y sobrescribe esos métodos, la lógica del hijo intentará ejecutarse antes de que el hijo haya terminado de inicializarse, causando posibles NullPointerException.
- **Solución Propuesta (Refactorización):** Ya que estos paneles y diálogos no están diseñados para ser heredados por otras clases en nuestro proyecto, se marcarán todas estas clases gráficas como final. Esto le garantiza al compilador (y al PMD) que nadie podrá extenderlas, haciendo que la llamada a sus métodos en el constructor sea 100% segura.

Análisis de Hallazgos: Paquete domain (Lógica de Negocio)

```
10  /**
11   * Guarda el estado completo de la partida usando serialización de objetos en disco.
12   */
13   public static void save(File file, DOPOGame game) throws DopeException {
14       try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream(file))) {
15           oos.writeObject(game);
16       } catch (IOException e) {
17           throw new DopeException("Error al guardar la partida por serialización binaria: " + e.getMessage());
18       }
19   }
20
21   /**
22   * Abre y restaura el estado completo de la partida usando deserialización de objetos desde el disco.
23   */
24   public static DOPOGame open(File file) throws DopeException {
25       try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(file))) {
26           return (DOPOGame) ois.readObject();
27       } catch (IOException | ClassNotFoundException e) {
28           throw new DopeException("Error al cargar la partida por serialización binaria: " + e.getMessage());
29       }
30   }
31 }
32
```

Regla Violada: AvoidFileStream (2 Violaciones en DOPOPersistence.java)

- **Explicación del Error:** El código está usando `new FileInputStream()` y `new FileOutputStream()` para el sistema de guardado y carga. PMD marca esto como una mala práctica moderna porque esas clases antiguas de Java (IO) tienen problemas de rendimiento
- **Solución Propuesta (Refactorización):** Se refactorizará el código para usar un paquete más modernos. Se reemplazarán por `Files.newInputStream()` y `Files.newOutputStream()`, los cuales son mucho más eficientes y seguros.

```
11 private LevelLoader() {}
12
13 private static final Map<String, LevelAction> commandRegistry = new HashMap<>();
14
```

Regla Violada: FieldNamingConventions (1 Violación en LevelLoader.java)

- **Explicación del Error:** En Java, existen convenciones estrictas de nombrado. El PMD detectó que la variable `commandRegistry` está declarada como `private static final`. Al ser estática y final, Java la considera una **Constante**. Las constantes no deben usar *camelCase*, sino **SCREAMING_SNAKE_CASE**.
- **Solución Propuesta (Refactorización):** Renombrar la variable de `commandRegistry` a `COMMAND_REGISTRY` en todo el archivo.

```

8  * Cargador de los niveles del juego
9  */
10 ClassWithOnlyPrivateConstructorsShouldBeFinal: This class has only private constructors and may be final
11  private LevelLoader() {}
12
13  private static final Map<String, LevelAction> commandRegistry = new HashMap<>();
14
15  @FunctionalInterface
16  private interface LevelAction {
17      void execute(String[] parts, Board board, Map<String, PlayerSkin> skinRegistry) throws Exception;
18  }
19

```

Regla Violada: ClassWithOnlyPrivateConstructorsShouldBeFinal (1 Violación en LevelLoader.java)

- **Explicación del Error:** LevelLoader es una clase de utilidad estática, por lo cual tiene su constructor bloqueado (private) para que nadie la instancie. Sin embargo, si una clase no puede ser instanciada, lógicamente tampoco debería poder ser heredada. PMD exige que esto sea explícito en la firma de la clase.
- **Solución Propuesta (Refactorización):** Agregar el modificador final a la declaración de la clase (public final class LevelLoader).

```

165
166  /**
167   * Define qué ocurre si otro objeto choca contra el jugador
168   * @param other El objeto causante del choque
169   */
170  @Override
171  public void applyCollisionEffect(Interactable other) {
172      // Player no aplica efectos a otros por defecto
173  }
174

```

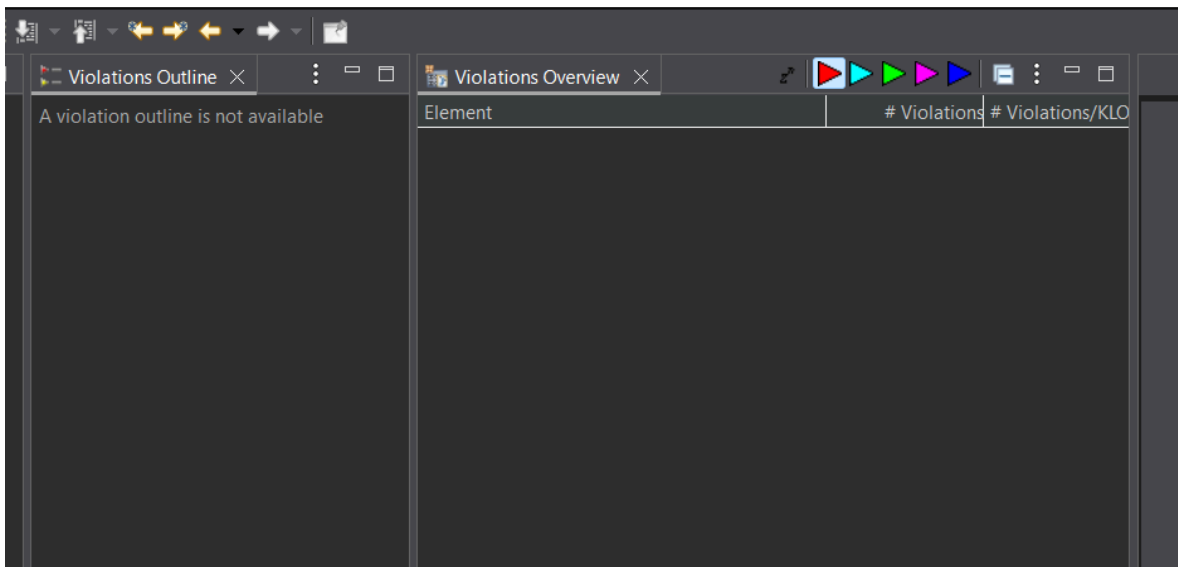
Regla Violada: EmptyMethodInAbstractClassShouldBeAbstract (1 Violación en Player.java)

- **Explicación del Error:** En una clase marcada como abstract (como Player), el objetivo es definir un contrato. El PMD detectó que el método applyCollisionEffect tiene las llaves vacías. Al estar vacío en una clase abstracta, PMD sugiere que es mejor declararlo directamente como abstract (sin llaves) para obligar a los hijos a implementarlo.

- **Solución Propuesta (Refactorización):** Dado que la interfaz Interactable ya le provee el contrato a Player, simplemente se borrará este método vacío de Player.java, delegando la responsabilidad directamente al polimorfismo del diseño.

Informe final de Análisis Estático Final (PMD)

Se realizaron los cambios explicados anteriormente para que el PMD no marcara errores rojos, eliminando así los de mayor prioridad.



Element	# Violations	# Violations/KLOC	# Violations/Method	Project
> test	549	N/A	N/A	HardestGame-V2
> presentation	621	497.6	6.90	HardestGame-V2
> VictoryDialog.java	28	160.0	28.00	HardestGame-V2
> LawOfDemeter	1	5.7	1.00	HardestGame-V2
> LocalVariableCouldBeFinal	18	102.9	18.00	HardestGame-V2
> MethodArgumentCouldBeFinal	1	5.7	1.00	HardestGame-V2
> AvoidLiteralsInIfCondition	1	5.7	1.00	HardestGame-V2
> CommentRequired	2	11.4	2.00	HardestGame-V2
> NcssCount	1	5.7	1.00	HardestGame-V2
> MissingSerialVersionUID	1	5.7	1.00	HardestGame-V2
> UnusedPrivateField	1	5.7	1.00	HardestGame-V2
> AvoidDeeplyNestedIfStmts	1	5.7	1.00	HardestGame-V2
> EmptyCatchBlock	1	5.7	1.00	HardestGame-V2
> V2GUIL.java	143	1007.0	2.92	HardestGame-V2
> SkinSelectionPanel.java	42	477.3	10.50	HardestGame-V2
> SettingsPanel.java	15	555.6	7.50	HardestGame-V2
> LocalVariableCouldBeFinal	5	185.2	2.50	HardestGame-V2
> MethodArgumentCouldBeFinal	2	74.1	1.00	HardestGame-V2
> CommentRequired	3	111.1	1.50	HardestGame-V2
> CallSuperInConstructor	1	37.0	0.50	HardestGame-V2
> ImmutableField	1	37.0	0.50	HardestGame-V2
> MissingSerialVersionUID	1	37.0	0.50	HardestGame-V2
> UnusedPrivateField	1	37.0	0.50	HardestGame-V2
> ShortVariable	1	37.0	0.50	HardestGame-V2
> PlayPanel.java	18	473.7	9.00	HardestGame-V2
> MainMenuPanel.java	20	487.8	10.00	HardestGame-V2
> UselessParentheses	1	24.4	0.50	HardestGame-V2
> LocalVariableCouldBeFinal	9	219.5	4.50	HardestGame-V2
> MethodArgumentCouldBeFinal	2	48.8	1.00	HardestGame-V2
> CommentRequired	3	73.2	1.50	HardestGame-V2
> CallSuperInConstructor	1	24.4	0.50	HardestGame-V2
> MissingSerialVersionUID	1	24.4	0.50	HardestGame-V2
> ImmutableField	1	24.4	0.50	HardestGame-V2
> UnusedPrivateField	1	24.4	0.50	HardestGame-V2
> ShortVariable	1	24.4	0.50	HardestGame-V2

Tras realizar el análisis estático con la herramienta PMD, se observan alertas de prioridad baja (verdes y morados, algunas azules), por lo tanto, se omitieron ya que no representan un problema crítico.